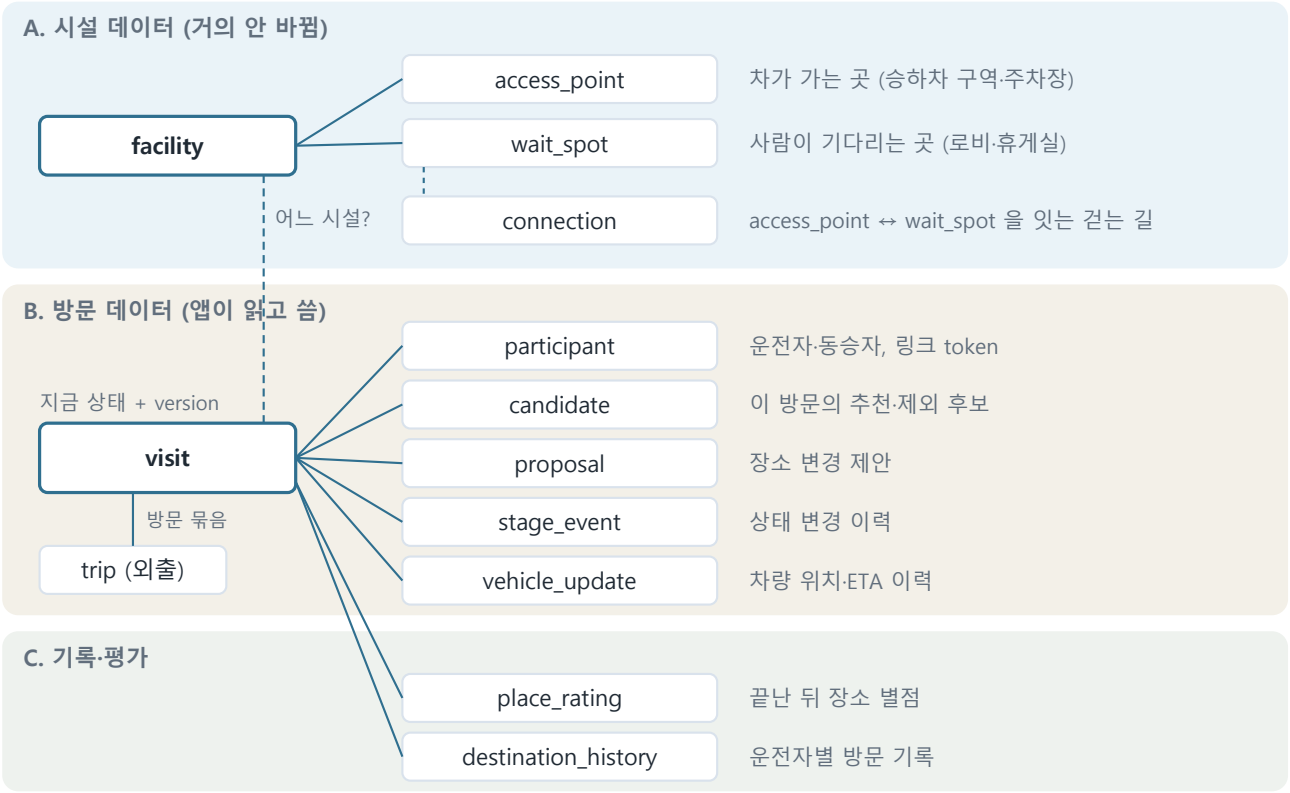
마중 DB 쉽게 이해하기

가족 픽업 앱 '마중'의 SQLite 데이터베이스 · 테이블 13개, 뷰 1개 · 2026-10-29

이 DB가 기억하는 것

아빠가 차로 쇼핑몰에 엄마를 데리러 가는 상황을 떠올려 보세요. DB는 세 가지를 기억해요.

- **시설**: 쇼핑몰의 어디에 차를 댈 수 있고, 어디서 기다릴 수 있고, 그 사이를 어떻게 걸어가는지
- **방문**: 오늘 이 픽업이 지금 어떤 상태인지, 누가 참여했고 어떤 장소를 추천했는지
- **기록**: 끝난 뒤의 평가와 방문 기록 (다음 추천에 씀)



선은 '하나 → 여러 개' 관계예요. 예를 들어 시설 1곳에 승하차 지점이 여러 개 있어요. 점선은 '어느 시설에 가는 방문인지'를 가리켜요.

꼭 알아야 할 아이디어 4가지

아이디어	쉽게 말하면	어디에 있나
후보 = 차 대는 곳 + 기다리는 곳 + 걷는 길	운전자는 승하차 지점을, 동승자는 대기 장소를 봐요. 둘을 잇는 길이 유모차·계단·비 조건을 정해요.	access_point, wait_spot, connection
지금 상태와 이력을 나눔	화면은 visit 한 줄만 읽으면 돼요. 무슨 일이 있었는지는 이력 테이블에 쌓여요.	visit / stage_event, vehicle_update
version으로 충돌 방지	고칠 때 '내가 본 번호'를 같이 보내요. 번호가 다르면 누가 먼저 고친 거라 거절해요.	visit.version
실수는 DB가 막음	없는 모드, 별점 6점, 두 번 누르기, 제안 2개 동시 등록, 같은 용도 1순위 2개, 엉뚱한 후보를 약속 장소로 저장하면 DB가 에러를 내요.	CHECK, UNIQUE, 외래키, 트리거

열 이름 읽는 법

모양	뜻	예
id	그 테이블의 번호	facility.id = 1
무엇_id	다른 테이블의 번호를 가리킴 (외래키)	facility_id = 1 → 시설 1번
무엇_at	시각 (UTC, 'YYYY-MM-DD HH:MM:SS')	created_at
has_ / is_ / _ok, 그 밖의 예·아니오	1 = 예, 0 = 아니오	has_roof = 1
무엇_json	JSON 글자로 저장	reasons_json = ["지붕 있음"]
비어 있음 (NULL)	값이 없음. 열마다 뜻이 정해져 있음	stop_limit_min NULL = 제한 없음

테이블별 열 설명

예시 값은 테스트 데이터(예시 쇼핑몰, visit-demo-001)에서 가져왔어요.

A. 시설 데이터

facility · 시설

시설 하나 = 한 줄. 쇼핑몰, 병원처럼 픽업하러 가는 장소예요.

열	뜻	예시
id	시설 번호	1
name	시설 이름	예시 쇼핑몰
address	주소	경기도 ○○시 ○○로 1
lat / lng	시설 중심 좌표 (위도 / 경도)	37.5400 / 127.2200
created_at	등록한 시각 (자동)	2026-10-29 08:26:44

access_point · 차가 가는 곳

운전자 화면에 보이는 지점이에요. 내비 목적지가 여기 좌표예요.

열	뜻	예시
id	지점 번호	1
facility_id	어느 시설의 지점인지	1 → 예시 쇼핑몰
type	curb = 승하차 구역, parking = 주차장	curb
name	지점 이름	서문 승하차 구역
lat / lng	내비에 넣는 좌표	37.5402 / 127.2190
has_roof	지붕 있음 (1/0)	북문 = 1
dropoff_allowed	승하차 허용. 0이면 후보에서 빠짐	동문 = 0
indoor	실내 (1/0)	B2 주차장 = 1
elevator_nearby	엘리베이터 가까움 (1/0)	북문 = 1
stop_limit_min	정차 제한(분). 비어 있으면 제한 없음	3
vehicle_notes	운전자 참고 메모	우회전 진입, 정차 3분
active	사용 중 (0이면 숨김)	1

wait_spot · 사람이 기다리는 곳

동승자 화면에 보이는 지점이에요.

열	뜻	예시
id	대기 장소 번호	3
facility_id	어느 시설인지	1
name	이름	2층 가족휴게실
floor	층	2F
indoor	실내 (1/0)	1
has_seating	의자 있음 (1/0)	1
description	설명	수유실·기저귀 교환대
active	사용 중 (0이면 숨김)	1

connection · 걷는 길

대기 장소에서 승하차 지점까지 걸어가는 길이에요. 같은 두 지점은 한 번만 연결할 수 있어요.

열	뜻	예시
id	길 번호	3
access_point_id	도착하는 승하차 지점	2 → 북문 승하차 구역
wait_spot_id	출발하는 대기 장소	2 → 1층 북문 로비
walk_minutes	걷는 시간(분)	1
stroller_ok	유모차로 갈 수 있음 (1/0)	1
no_stairs	계단 없음 (1/0)	가족휴게실→서문 = 0
covered	비 안 맞고 갈 수 있음 (1/0)	1
description	설명	북문 로비 → 북문 구역, 지붕 연결

B. 방문 데이터

trip · 외출

한 번의 외출 = 한 줄. 내려주기 → 다시 만나기(시나리오 B), 픽업 → 다른 곳 주차(시나리오 C)처럼 방문이 여러 개일 때 묶어요.

열	뜻	예시
id	외출 ID (서버가 만듦)	trip-b-001
driver_device_id	운전자 기기	device-car-001
share_token	외출 전체 공유 링크. 열면 지금 진행 중인 방문을 보여줌 (중복 불가)	tok-trip-b
created_at	만든 시각	

visit · 방문 한 건

픽업 한 번 = 한 줄. 화면에 보이는 '지금 상태'는 모두 여기 있어요.

열	뜻	예시
id	방문 ID (서버가 만듦)	visit-demo-001
trip_id	어느 외출에 속하는지 (trip). 방문 하나뿐이면 비어 있음	trip-b-001
facility_id	어느 시설에 가는지	1
mode	pickup = 데리러, dropoff = 내려주기, parking = 주차	pickup
status	큰 진행 상태: created → moving → arrived → boarded → ended (또는 cancelled)	created
current_stage	세부 단계 (departed, pickup_started, dropoff_done, parked, arrival_imminent, cancelled 등)	created
confirmed_candidate_id	약속한 만남·하차 장소 (purpose = meet 후보)	2 → 서문
parking_candidate_id	주차할 곳 (purpose = parking 후보, 모든 모드)	비어 있음
driver_device_id	운전자 기기 ID (로그인 대신)	device-car-001
final_destination_json	태운 뒤 갈 곳	{"name":"집", ...}
weather_json	추천할 때의 날씨	{"condition":"rain", ...}
last_lat / last_lng	차의 마지막 위치	비어 있음 = 위치 모름
last_eta_sec	도착까지 남은 시간(초)	240
last_nav_result	내비 설정 결과: set / failed	set
last_vehicle_at	차 정보를 마지막으로 받은 시각	2026-10-29 09:00
version	고칠 때마다 1씩 올라가는 번호 (충돌 방지)	1
expires_at	링크 만료 시각	만든 뒤 하루
created_at / updated_at	만든 시각 / 마지막 수정 시각 (자동)	

participant · 참여자

방문에 참여한 사람. 운전자는 방문당 1명만 있을 수 있어요.

열	뜻	예시
id	참여자 번호	2
visit_id	어느 방문인지	visit-demo-001
role	driver = 운전자, passenger = 동승자	passenger
token	링크·QR에 들어가는 값 (중복 불가)	tok-passenger-001
display_name	화면에 보이는 이름	엄마
conditions_json	필요 조건	{"stroller":true, "avoid_stairs":true, ...}
current_wait_spot_id	지금 가까운 대기 장소	3 → 2층 가족휴게실
status	none / preparing / moving_to_lobby / at_lobby	preparing
expected_ready_at	준비될 예정 시각 (표시용)	비어 있음
joined_at / last_seen_at	링크로 들어온 시각 / 마지막 접속	

candidate · 추천 후보

이 방문에서 추천하거나 뺀 후보예요. 시설 정보가 바뀌어도 과거 기록이 그대로 남도록 방문마다 복사해 둬요.

열	뜻	예시
id	후보 번호	1
visit_id	어느 방문의 후보인지	visit-demo-001
purpose	meet = 만남·하차 지점, parking = 주차 자리	meet
access_point_id	승하차 지점	2 → 북문
wait_spot_id	대기 장소. 추천된 meet 후보는 꼭 있어야 함, 주차 후보는 비어도 됨	2 → 1층 북문 로비
connection_id	둘을 잇는 길	3
rank	purpose별 추천 순위. 1이 최우선, 비어 있으면 제외. 같은 순위 중복 불가	1
score	점수 (0~1)	0.92
reasons_json	추천 이유	["지붕 있음","도보 1분"]
excluded_reason	뺀 이유 (비어 있으면 추천 후보)	승하차 금지 구역
created_at	만든 시각	

proposal · 장소 변경 제안

'여기로 바꾸자'는 제안. 방문당 대기 중인 제안은 1개만 있을 수 있어요.

열	뜻	예시
id	제안 번호	1
visit_id	어느 방문인지	visit-demo-001
candidate_id	바꾸자고 한 후보	1 → 북문
proposer_id	제안한 사람 (participant)	2 → 엄마
reason	rain / closer / stroller / crowded / other	rain
reason_text	직접 쓴 이유	
status	pending(대기) / accepted / rejected / expired	pending
base_version	제안할 때 본 visit.version	1
request_id	요청 고유값 (두 번 눌러도 한 번만)	req-proposal-001
responder_id	수락·거절한 사람	1 → 아빠
responded_via	touch / voice	
created_at / responded_at	제안 시각 / 응답 시각	

stage_event · 상태 변경 이력

visit의 단계가 바뀔 때마다 한 줄씩 쌓여요. 지우거나 고치지 않아요.

열	뜻	예시
id	이력 번호	1
visit_id	어느 방문인지	visit-demo-001
stage	바뀐 단계 (함께 출발은 departed, 취소는 cancelled)	pickup_started
actor_id	누가 바꿨는지 (participant)	1 → 아빠
source	manual = 버튼, sensor = 차량 센서, system = 서버	manual
request_id	요청 고유값 (중복 방지)	req-stage-001
created_at	기록 시각	

vehicle_update · 차량 위치 이력

차가 보낸 위치·ETA를 받을 때마다 한 줄. 최신값은 visit.last_* 에도 복사해요.

열	뜻	예시
id	이력 번호	2
visit_id	어느 방문인지	visit-demo-001
lat / lng	차 위치 (비어 있으면 위치 없음, 예: 지하주차장)	37.5405 / 127.2195
eta_sec	도착까지 남은 시간(초)	240
nav_result	내비 설정: set / failed	set
nav_target_access_point_id	내비가 향하는 지점	2 → 북문
recorded_at	받은 시각	

C. 기록·평가

place_rating · 장소 평가

방문이 끝난 뒤 만난 장소에 매긴 별점이에요.

열	뜻	예시
id	평가 번호	
visit_id / candidate_id	어느 방문의 어느 장소인지	
rating	별점 1~5 (그 밖의 값은 DB가 거부)	5
tags_json	태그	["유모차 편함"]
comment	한 줄 의견	
created_at	평가 시각	

destination_history · 방문 기록

운전자별로 어디에 어떤 장소로 다녀왔는지. 다음 추천에 써요.

열	뜻	예시
id	기록 번호	1
driver_device_id	운전자 기기	device-car-001
facility_id	다녀온 시설	1
mode	모드	pickup
last_candidate_id	마지막으로 쓴 장소	1 → 북문
visited_at	다녀온 시각	

v_visit_card (뷰)

테이블이 아니라 저장된 SELECT예요. visit, facility, 확정된 candidate, access_point, wait_spot, connection, 동승자, 대기 중 제안을 합쳐 방문 카드 한 장에 필요한 값을 한 줄로 보여줘요. 방문 카드(W01)와 차량 화면(C06)이 이 뷰 하나만 읽어요.

시나리오 직접 돌려보기

테스트 방문 visit-demo-001로 '비 오는 날 픽업'을 처음부터 끝까지 따라가요. 각 단계의 SQL을 sqlite3에 복사해 붙여 넣으면 돼요. 이 문서의 SQL은 모두 실제로 실행해서 확인했어요.

준비: DB를 처음 상태로 만들기 (PowerShell)

```
cd majung-db                    # 저장소 폴더
$env:Path += ";$env:LOCALAPPDATA₩Microsoft₩WinGet₩Packages₩SQLite.SQLite_Microsoft.Winget.Source_8wekyb3d8bbwe"
chcp 65001                      # 한글이 깨질 때

Remove-Item majung.db*          # 기존 DB 지우기 (처음부터 다시 할 때만)
sqlite3 majung.db ".read majung_db/01_schema.sql" ".read majung_db/02_seed_facility.sql" ".read majung_db/03_demo_visit.sql"
sqlite3 majung.db ".read majung_db/05_seed_scenarios.sql"    # 시나리오 A·B·C용 시설 2 추가

sqlite3 majung.db               # sqlite3 들어가기
```

sqlite3 안에서 먼저 실행:

```
.headers on
.mode column
PRAGMA foreign_keys = ON;       -- 외래키 검사 켜기 (안 켜면 실험 5가 안 막힘)
```

다 끝나면 **.quit**으로 나가세요. 켜 두면 파일이 잠겨서 지우거나 바꿀 수 없어요. PowerShell에서는 'sqlite3 majung.db < 파일' 방식이 안 되니 .read를 써요.

한눈에 보는 흐름

단계	누가	바뀌는 것
1~3	엄마·아빠	보기만 함
4	아빠	약속 장소 서문 → 북문, version 2
5	아빠	픽업 시작, version 3
6~7	차	위치·ETA 기록, '나오세요' 판정
8	엄마	로비 도착
9	아빠	방문 종료, version 4

1. 방문 카드 보기

엄마가 링크를 열면 보이는 화면이에요. 지금은 서문에서 만나기로 했고, 제안 1개가 기다리고 있어요.

```
SELECT mode, current_stage, version, access_point_name, wait_spot_name,
       walk_minutes, passenger_status, pending_proposal_id
  FROM v_visit_card WHERE visit_id = 'visit-demo-001';
```

→ 서문 승하차 구역 / 1층 서문 로비, version 1, pending_proposal_id 1

2. 추천 후보 보기

추천된 곳은 순위대로, 뺀 곳은 맨 아래에 이유와 함께 나와요.

```
SELECT c.purpose, c.rank, ap.name AS access_point, ws.name AS wait_spot,
       c.reasons_json, c.excluded_reason
  FROM candidate c
  JOIN access_point ap ON ap.id = c.access_point_id
  LEFT JOIN wait_spot ws ON ws.id = c.wait_spot_id
 WHERE c.visit_id = 'visit-demo-001'
 ORDER BY c.purpose, c.rank IS NULL, c.rank;
```

→ 1순위 북문, 2순위 서문, 동문은 '승하차 금지 구역'으로 제외

3. 엄마의 제안 보기

비가 와서 엄마가 북문으로 바꾸자고 했어요. 아빠 차 화면에 뜨는 알림이에요.

```
SELECT pr.id, pr.reason, pr.status, p.display_name AS proposer,
       ap.name AS proposed_place
  FROM proposal pr
  JOIN participant p    ON p.id  = pr.proposer_id
  JOIN candidate c      ON c.id  = pr.candidate_id
  JOIN access_point ap ON ap.id = c.access_point_id
 WHERE pr.visit_id = 'visit-demo-001' AND pr.status = 'pending';
```

→ 1 | rain | pending | 엄마 | 북문 승하차 구역

4. 아빠가 제안 수락

'내가 본 version이 1일 때만 바꿔라'가 핵심이에요. 그사이 누가 먼저 바꿨으면 0줄이 바뀌고, 서버는 충돌로 처리해요.

```
BEGIN;
UPDATE visit
   SET confirmed_candidate_id = 1, version = version + 1
 WHERE id = 'visit-demo-001' AND version = 1;
SELECT changes();   -- 1이면 성공. 0이면 충돌이니 ROLLBACK;
UPDATE proposal
   SET status = 'accepted', responder_id = 1, responded_at = datetime('now')
 WHERE id = 1 AND status = 'pending';
COMMIT;
```

→ changes() = 1. 1번을 다시 실행하면 북문 / 1층 북문 로비, version 2, 제안 없음

5. 픽업 시작

이력(stage_event)에 한 줄 쌓고, 지금 상태(visit)를 바꿔요. 둘을 한 트랜잭션으로 묶어요.

```
BEGIN;
INSERT INTO stage_event (visit_id, stage, actor_id, source, request_id)
VALUES ('visit-demo-001', 'pickup_started', 1, 'manual', 'req-stage-001');
UPDATE visit
   SET current_stage = 'pickup_started', status = 'moving', version = version + 1
 WHERE id = 'visit-demo-001' AND version = 2;
COMMIT;
```

→ current_stage = pickup_started, status = moving, version 3

6. 차 위치·ETA 받기

차가 4분(240초) 거리에 있어요. 이력도 남기고 최신값도 visit에 같이 써요.

```
BEGIN;
INSERT INTO vehicle_update (visit_id, lat, lng, eta_sec, nav_result, nav_target_access_point_id)
VALUES ('visit-demo-001', 37.5405, 127.2195, 240, 'set', 2);
UPDATE visit
   SET last_lat = 37.5405, last_lng = 127.2195, last_eta_sec = 240,
       last_nav_result = 'set', last_vehicle_at = datetime('now')
 WHERE id = 'visit-demo-001';
COMMIT;
```

→ vehicle_update가 2줄이 되고, 방문 카드에 last_eta_sec 240이 보여요

7. '이제 나오세요' 판정

차가 도착하는 시간이 엄마가 걷는 시간보다 짧아지면 나오라고 알려요.

```
SELECT last_eta_sec, walk_minutes * 60 AS walk_sec,
       CASE WHEN last_eta_sec IS NULL THEN 'no_location'
            WHEN last_eta_sec <= walk_minutes * 60 THEN 'go_now'
            ELSE 'wait' END AS passenger_instruction
  FROM v_visit_card WHERE visit_id = 'visit-demo-001';
```

→ 240초 > 60초라서 wait. 아래처럼 ETA를 50초로 바꾸고 다시 실행하면 go_now

7-1. 차가 더 가까워짐 (연습용)

6번처럼 하는 게 정석이지만, 연습할 때는 값만 바꿔 봐도 돼요. 바꾼 뒤 7번을 다시 실행하세요.

```
UPDATE visit SET last_eta_sec = 50 WHERE id = 'visit-demo-001';
```

→ 7번 결과가 go_now로 바뀜

8. 엄마가 로비 도착

동승자는 로그인 대신 링크의 token으로 구분해요.

```
UPDATE participant
   SET status = 'at_lobby', last_seen_at = datetime('now')
 WHERE token = 'tok-passenger-001';

SELECT role, display_name, status FROM participant;
```

→ 엄마(passenger)의 status = at_lobby

9. 탑승 끝, 방문 종료

방문을 닫고 다음 추천에 쓰도록 기록을 남겨요.

```
BEGIN;
UPDATE visit SET status = 'ended', current_stage = 'ended', version = version + 1
 WHERE id = 'visit-demo-001';
INSERT INTO destination_history (driver_device_id, facility_id, mode, last_candidate_id)
SELECT driver_device_id, facility_id, mode, confirmed_candidate_id
  FROM visit WHERE id = 'visit-demo-001';
COMMIT;

SELECT status, current_stage, version FROM visit;
SELECT * FROM destination_history;
```

→ ended / ended / version 4, 방문 기록 1줄 (last_candidate_id = 1, 북문)

DB가 막아주는 실수 (9단계 뒤에 해보기)

앱 코드가 실수해도 DB가 마지막에 막아줘요. 에러가 나면 성공이에요.

실험 1. 늦게 온 수락 (version 충돌)

누군가 이미 바꿨는데 옛날 화면(version 1)을 보고 다시 수락하는 상황이에요.

```
UPDATE visit SET confirmed_candidate_id = 2, version = version + 1
 WHERE id = 'visit-demo-001' AND version = 1;
SELECT changes();
```

→ changes() = 0. 아무것도 안 바뀜 → 서버는 '다른 사람이 먼저 바꿨어요'(409)로 응답

실험 2. 버튼 두 번 누르기 (request_id)

5번의 INSERT를 그대로 한 번 더 실행해요.

```
INSERT INTO stage_event (visit_id, stage, actor_id, source, request_id)
VALUES ('visit-demo-001', 'pickup_started', 1, 'manual', 'req-stage-001');
```

→ 에러: UNIQUE constraint failed: stage_event.request_id

실험 3. 없는 모드 넣기 (CHECK)

mode는 pickup, dropoff, parking 세 가지만 돼요.

```
INSERT INTO visit (id, facility_id, mode, driver_device_id, expires_at)
VALUES ('visit-test', 1, 'walk', 'device-x', datetime('now', '+1 day'));
```

→ 에러: CHECK constraint failed: mode IN ('pickup', 'dropoff', 'parking')

실험 4. 대기 중 제안 2개 (부분 유니크 인덱스)

한 방문에 pending 제안이 이미 있으면 새 제안을 못 넣어요. 첫 줄은 들어가고 둘째 줄에서 막혀요.

```
INSERT INTO proposal (visit_id, candidate_id, proposer_id, base_version, request_id)
VALUES ('visit-demo-001', 2, 1, 4, 'req-test-a');
INSERT INTO proposal (visit_id, candidate_id, proposer_id, base_version, request_id)
VALUES ('visit-demo-001', 1, 2, 4, 'req-test-b');
```

→ 에러: UNIQUE constraint failed: proposal.visit_id

실험 5. 없는 시설에 지점 넣기 (외래키)

시설 99번은 없어요. PRAGMA foreign_keys = ON 을 먼저 실행해야 막혀요.

```
INSERT INTO access_point (facility_id, type, name, lat, lng)
VALUES (99, 'curb', '없는 시설의 지점', 0, 0);
```

→ 에러: FOREIGN KEY constraint failed

실험 6. 1순위 후보 2개 (용도별 순위)

이 방문의 만남(meet) 1순위는 이미 북문이에요. 같은 용도에 1순위를 하나 더 넣으면 막혀요. 주차(parking) 1순위는 따로라서 넣을 수 있어요.

```
INSERT INTO candidate (visit_id, purpose, access_point_id, wait_spot_id, connection_id, rank)
VALUES ('visit-demo-001', 'meet', 1, 1, 1, 1);
```

→ 에러: UNIQUE constraint failed: candidate.visit_id, candidate.purpose, candidate.rank

실험 7. 만남 후보를 주차 자리로 (트리거)

1번 후보는 북문 만남(meet) 후보예요. 주차 자리(parking_candidate_id)에 넣으면 트리거가 막아요. purpose는 다른 테이블(candidate)에 있어서 CHECK로는 못 막고 트리거로 막아요.

```
UPDATE visit SET parking_candidate_id = 1 WHERE id = 'visit-demo-001';
```

→ 에러: parking_candidate_id에는 이 방문의 parking 후보만 넣을 수 있어요

실험 8. 다른 방문의 후보 넣기 (트리거)

새 방문을 하나 만들고, 데모 방문의 후보(1번)를 약속 장소로 넣어 봐요. purpose는 meet로 맞지만 다른 방문의 후보라 막혀요.

```
INSERT INTO visit (id, facility_id, mode, driver_device_id, expires_at)
VALUES ('visit-test-002', 1, 'pickup', 'device-x', datetime('now', '+1 day'));
UPDATE visit SET confirmed_candidate_id = 1 WHERE id = 'visit-test-002';
```

→ 에러: confirmed_candidate_id에는 이 방문의 meet 후보만 넣을 수 있어요

시나리오 A·B·C 돌려보기

기획서의 세 시나리오를 DB에서 처음부터 끝까지 돌려봐요. SQL은 majung_db/06_scenarios_abc.sql에 그대로 있고, 아래 내용은 그 파일에서 가져왔어요. 모든 단계를 실제로 실행해서 version과 결과를 확인했어요.

	A. 목적지에서 픽업	B. 함께 출발	C. 픽업 후 다른 장소로
방문 수	1개 (pickup)	2개 (dropoff → pickup)	2개 (pickup → parking)
묶는 법	없음	trip-b-001 (링크 tok-trip-b)	trip-c-001 (링크 tok-trip-c)
주차	만난 뒤 같은 시설에 주차	내려준 뒤 주차, 주차장에서 다시 만남	다른 시설(미술관)에 날씨 보고 주차
핵심 열	candidate.purpose, parking_candidate_id	trip.share_token, departed, source = sensor	weather_json, last_eta_sec

실행 방법

- '준비' 단계로 DB를 새로 만든 뒤(05까지 포함) sqlite3에 들어가요.
- 한 단계씩: 아래 SQL을 순서대로 붙여 넣어요. 순서를 건너뛰면 version 조건이 안 맞아 0줄이 바뀌어요.
- 한 번에: **.read majung_db/06_scenarios_abc.sql** 을 실행하면 세 시나리오가 끝까지 돌아가요.
- A·B·C 데이터는 visit-a-001, visit-b-drop처럼 따로 만들어서, 앞의 visit-demo-001과 섞이지 않아요.

시나리오 A. 운전자가 목적지에서 가족 픽업

아빠가 집에서 출발해 쇼핑몰에 있는 엄마를 데리러 가요. 맑은 날이에요.

단계	하는 일	결과
A-1	출발 전: 목적지 입력, 만남·주차 장소 추천	만남 후보: 서문 1순위, 북문 2순위, 동문 제외 / 주차 후보: B2 1순위
A-2	목적지 결정, 동승자에게 링크 전달	version 2, 엄마 링크(tok-a-passenger)로 열면 서문 승하차 구역 / 1층 서문 로비
A-3	운전자 이동, 동승자가 위치 확인	방문 카드에 status moving, 남은 시간 900초(15분)
A-4	만남: 운전자가 확인	status boarded, stage_event 3줄 (pickup_started, arrived, boarded)
A-5	주차 장소 추천, 주차 완료 저장	지하주차장 B2 입구에 주차, status ended, version 6, 방문 기록 1줄

A-1. 출발 전: 목적지 입력, 만남·주차 장소 추천

방문을 만들고 추천 후보를 저장해요. 만남 후보(승하차 구역 + 로비)와 주차 후보(주차장)가 같이 들어가고, purpose로 구분해요. 순위도 purpose별로 따로 매겨요.

```
BEGIN;
INSERT INTO visit (id, facility_id, mode, driver_device_id, weather_json, expires_at)
VALUES ('visit-a-001', 1, 'pickup', 'device-car-001',
        '{"condition":"clear","temp_c":18}', datetime('now', '+1 day'));
INSERT INTO participant (visit_id, role, token, display_name)
VALUES ('visit-a-001', 'driver', 'tok-a-driver', '아빠');
INSERT INTO candidate (visit_id, purpose, access_point_id, wait_spot_id, connection_id, rank, score, reasons_json, excluded_reason)
VALUES
  ('visit-a-001', 'meet',    1, 1,    1,    1,    0.88, '["맑음","가장 가까움","도보 2분"]', NULL),
  ('visit-a-001', 'meet',    2, 2,    3,    2,    0.80, '["지붕 있음","도보 1분"]', NULL),
  ('visit-a-001', 'meet',    3, NULL, NULL, NULL, NULL, '승하차 금지 구역'),
  ('visit-a-001', 'parking', 4, NULL, NULL, 1,    0.90, '["실내 주차","엘리베이터 가까움"]', NULL);
COMMIT;

SELECT c.purpose, c.rank, ap.name AS access_point, ws.name AS wait_spot, c.excluded_reason
  FROM candidate c
  JOIN access_point ap ON ap.id = c.access_point_id
  LEFT JOIN wait_spot ws ON ws.id = c.wait_spot_id
 WHERE c.visit_id = 'visit-a-001'
 ORDER BY c.purpose, c.rank IS NULL, c.rank;
```

→ 만남 후보: 서문 1순위, 북문 2순위, 동문 제외 / 주차 후보: B2 1순위

A-2. 목적지 결정, 동승자에게 링크 전달

아빠가 서문을 고르면 version 1 → 2. 엄마를 참여자로 넣으면 token이 생기고, 이 값이 링크에 들어가요.

```
UPDATE visit
   SET confirmed_candidate_id = (SELECT id FROM candidate WHERE visit_id = 'visit-a-001' AND access_point_id = 1),
       version = version + 1
 WHERE id = 'visit-a-001' AND version = 1;

INSERT INTO participant (visit_id, role, token, display_name, conditions_json, status)
VALUES ('visit-a-001', 'passenger', 'tok-a-passenger', '엄마', '{"stroller":true}', 'preparing');

SELECT p.display_name, v.visit_id, v.version, v.access_point_name, v.wait_spot_name
  FROM participant p JOIN v_visit_card v ON v.visit_id = p.visit_id
 WHERE p.token = 'tok-a-passenger';
```

→ version 2, 엄마 링크(tok-a-passenger)로 열면 서문 승하차 구역 / 1층 서문 로비

A-3. 운전자 이동, 동승자가 위치 확인

픽업 시작을 이력에 남기고 visit 상태를 바꿔요(version 3). 차 위치는 vehicle_update에 쌓고 최신값은 visit에 복사해요.

```
BEGIN;
INSERT INTO stage_event (visit_id, stage, actor_id, request_id)
VALUES ('visit-a-001', 'pickup_started', (SELECT id FROM participant WHERE token = 'tok-a-driver'), 'req-a-start');
UPDATE visit SET status = 'moving', current_stage = 'pickup_started', version = version + 1
 WHERE id = 'visit-a-001' AND version = 2;
INSERT INTO vehicle_update (visit_id, lat, lng, eta_sec, nav_result, nav_target_access_point_id)
VALUES ('visit-a-001', 37.5300, 127.2000, 900, 'set', 1);
UPDATE visit SET last_lat = 37.5300, last_lng = 127.2000, last_eta_sec = 900,
                 last_nav_result = 'set', last_vehicle_at = datetime('now')
 WHERE id = 'visit-a-001';
COMMIT;

SELECT status, current_stage, last_lat, last_lng, last_eta_sec
  FROM v_visit_card WHERE visit_id = 'visit-a-001';
```

→ 방문 카드에 status moving, 남은 시간 900초(15분)

A-4. 만남: 운전자가 확인

아빠가 '만났어요'를 누르면 도착·탑승 이력이 쌓이고 visit은 boarded(version 4)가 돼요.

```
BEGIN;
INSERT INTO stage_event (visit_id, stage, actor_id, request_id)
VALUES ('visit-a-001', 'arrived', (SELECT id FROM participant WHERE token = 'tok-a-driver'), 'req-a-arrived'),
       ('visit-a-001', 'boarded', (SELECT id FROM participant WHERE token = 'tok-a-driver'), 'req-a-boarded');
UPDATE visit SET status = 'boarded', current_stage = 'boarded', version = version + 1
 WHERE id = 'visit-a-001' AND version = 3;
COMMIT;

SELECT stage, created_at FROM stage_event WHERE visit_id = 'visit-a-001' ORDER BY id;
```

→ status boarded, stage_event 3줄 (pickup_started, arrived, boarded)

A-5. 주차 장소 추천, 주차 완료 저장

A-1에서 저장한 주차 후보 중 1순위를 골라 parking_candidate_id에 넣고, 주차 완료(version 5) 뒤 방문을 닫아요(version 6).

```
SELECT c.rank, ap.name, c.reasons_json
  FROM candidate c JOIN access_point ap ON ap.id = c.access_point_id
 WHERE c.visit_id = 'visit-a-001' AND c.purpose = 'parking' AND c.excluded_reason IS NULL
 ORDER BY c.rank;

BEGIN;
UPDATE visit
   SET parking_candidate_id = (SELECT id FROM candidate WHERE visit_id = 'visit-a-001' AND access_point_id = 4),
       current_stage = 'parked', version = version + 1
 WHERE id = 'visit-a-001' AND version = 4;
INSERT INTO stage_event (visit_id, stage, actor_id, request_id)
VALUES ('visit-a-001', 'parked', (SELECT id FROM participant WHERE token = 'tok-a-driver'), 'req-a-parked');
UPDATE visit SET status = 'ended', current_stage = 'ended', version = version + 1
 WHERE id = 'visit-a-001' AND version = 5;
INSERT INTO destination_history (driver_device_id, facility_id, mode, last_candidate_id)
SELECT driver_device_id, facility_id, mode, confirmed_candidate_id FROM visit WHERE id = 'visit-a-001';
COMMIT;

SELECT v.status, v.version, ap.name AS parked_at
  FROM visit v
  JOIN candidate c      ON c.id  = v.parking_candidate_id
  JOIN access_point ap ON ap.id = c.access_point_id
 WHERE v.id = 'visit-a-001';
```

→ 지하주차장 B2 입구에 주차, status ended, version 6, 방문 기록 1줄

시나리오 B. 운전자와 동승자가 함께 출발

비 오는 날, 아빠가 엄마를 쇼핑몰에 내려주고 주차해요. 나중에 주차장에서 다시 만나요. 내려주기와 만나기는 방문 2개로 나누고 trip으로 묶어요. 엄마는 외출 링크 하나로 끝까지 봐요.

단계	하는 일	결과
B-1	목적지 선택, 내려줄 곳·주차 자리 추천	만남(내려주기) 후보 북문 1순위, 주차 후보 B2 1순위
B-2	목적지 결정	북문 승하차 구역에 내리고 B2에 주차하기로 함
B-3	운전자 이동	status moving, current_stage departed, 남은 시간 1200초
B-4	동승자 내려주기 & 주차	current_stage parked, version 5, 이력: departed → dropoff_done(sensor) → parked(manual)
B-5	시간 보내기	visit-b-drop: ended, version 6
B-6	운전자가 먼저 주차장에 가서 만남 장소 선택	trip-b-001에 방문 2개 (dropoff ended, pickup created). 외출 링크 → visit-b-pick, 만남 장소 지하주차장 B2 입구 / 2층 가족휴게실
B-7	만나기	visit-b-pick: ended, version 3. 엄마 status at_lobby

B-1. 목적지 선택, 내려줄 곳·주차 자리 추천

외출(trip)을 먼저 만들고 공유 링크(share_token)를 붙여요. 엄마도 차에 타고 있으니 처음부터 참여자 2명이에요. 비가 와서 지붕 있는 북문과 실내 주차장이 1순위예요.

```
BEGIN;
INSERT INTO trip (id, driver_device_id, share_token)
VALUES ('trip-b-001', 'device-car-001', 'tok-trip-b');
INSERT INTO visit (id, trip_id, facility_id, mode, driver_device_id, weather_json, expires_at)
VALUES ('visit-b-drop', 'trip-b-001', 1, 'dropoff', 'device-car-001',
        '{"condition":"rain","temp_c":14}', datetime('now', '+1 day'));
INSERT INTO participant (visit_id, role, token, display_name, conditions_json)
VALUES ('visit-b-drop', 'driver',    'tok-b-driver',    '아빠', NULL),
       ('visit-b-drop', 'passenger', 'tok-b-passenger', '엄마', '{"stroller":true,"prefer_indoor":true}');
INSERT INTO candidate (visit_id, purpose, access_point_id, wait_spot_id, connection_id, rank, score, reasons_json, excluded_reason)
VALUES
  ('visit-b-drop', 'meet',    2, 2,    3,    1,    0.93, '["비","지붕 있음","유모차 실내 연결"]', NULL),
  ('visit-b-drop', 'meet',    1, 1,    1,    2,    0.55, '["가까움","비 맞음"]', NULL),
  ('visit-b-drop', 'meet',    3, NULL, NULL, NULL, NULL, '승하차 금지 구역'),
  ('visit-b-drop', 'parking', 4, NULL, NULL, 1,    0.95, '["비","실내 주차","유모차 전용 주차면"]', NULL);
COMMIT;

SELECT c.purpose, c.rank, ap.name, c.reasons_json
  FROM candidate c JOIN access_point ap ON ap.id = c.access_point_id
 WHERE c.visit_id = 'visit-b-drop' AND c.excluded_reason IS NULL
 ORDER BY c.purpose, c.rank;
```

→ 만남(내려주기) 후보 북문 1순위, 주차 후보 B2 1순위

B-2. 목적지 결정

내려줄 곳(confirmed_candidate_id)과 주차할 곳(parking_candidate_id)을 한 번에 정해요. version 1 → 2.

```
UPDATE visit
   SET confirmed_candidate_id = (SELECT id FROM candidate WHERE visit_id = 'visit-b-drop' AND access_point_id = 2),
       parking_candidate_id   = (SELECT id FROM candidate WHERE visit_id = 'visit-b-drop' AND access_point_id = 4),
       version = version + 1
 WHERE id = 'visit-b-drop' AND version = 1;

SELECT access_point_name AS dropoff_at, version FROM v_visit_card WHERE visit_id = 'visit-b-drop';
```

→ 북문 승하차 구역에 내리고 B2에 주차하기로 함

B-3. 운전자 이동

함께 타고 출발하니 pickup_started 대신 departed 단계를 남겨요(version 3).

```
BEGIN;
INSERT INTO stage_event (visit_id, stage, actor_id, request_id)
VALUES ('visit-b-drop', 'departed', (SELECT id FROM participant WHERE token = 'tok-b-driver'), 'req-b-departed');
UPDATE visit SET status = 'moving', current_stage = 'departed', version = version + 1
 WHERE id = 'visit-b-drop' AND version = 2;
INSERT INTO vehicle_update (visit_id, lat, lng, eta_sec, nav_result, nav_target_access_point_id)
VALUES ('visit-b-drop', 37.5200, 127.1800, 1200, 'set', 2);
UPDATE visit SET last_lat = 37.5200, last_lng = 127.1800, last_eta_sec = 1200,
                 last_nav_result = 'set', last_vehicle_at = datetime('now')
 WHERE id = 'visit-b-drop';
COMMIT;
```

→ status moving, current_stage departed, 남은 시간 1200초

B-4. 동승자 내려주기 & 주차

문이 열리면 차량 센서가 하차 완료를 알려요(source = sensor). 그다음 주차 완료. 단계마다 version이 올라 5가 돼요.

```
BEGIN;
INSERT INTO stage_event (visit_id, stage, actor_id, source, request_id)
VALUES ('visit-b-drop', 'dropoff_done', NULL, 'sensor', 'req-b-dropoff');
UPDATE visit SET status = 'arrived', current_stage = 'dropoff_done', version = version + 1
 WHERE id = 'visit-b-drop' AND version = 3;
INSERT INTO stage_event (visit_id, stage, actor_id, request_id)
VALUES ('visit-b-drop', 'parked', (SELECT id FROM participant WHERE token = 'tok-b-driver'), 'req-b-parked');
UPDATE visit SET current_stage = 'parked', version = version + 1
 WHERE id = 'visit-b-drop' AND version = 4;
COMMIT;

SELECT stage, source FROM stage_event WHERE visit_id = 'visit-b-drop' ORDER BY id;
```

→ current_stage parked, version 5, 이력: departed → dropoff_done(sensor) → parked(manual)

B-5. 시간 보내기

내려주기 방문은 여기서 닫아요(version 6). 각자 시간을 보내는 동안 DB에 새로 쌓이는 건 없어요.

```
BEGIN;
UPDATE visit SET status = 'ended', current_stage = 'ended', version = version + 1
 WHERE id = 'visit-b-drop' AND version = 5;
INSERT INTO destination_history (driver_device_id, facility_id, mode, last_candidate_id)
SELECT driver_device_id, facility_id, mode, parking_candidate_id FROM visit WHERE id = 'visit-b-drop';
COMMIT;
```

→ visit-b-drop: ended, version 6

B-6. 운전자가 먼저 주차장에 가서 만남 장소 선택

같은 trip으로 픽업 방문을 새로 만들어요. 차가 B2에 있으니 B2 입구 + 가족휴게실이 1순위예요. 엄마가 처음 받은 외출 링크(tok-trip-b)를 다시 열면 새 방문이 보여요.

```
BEGIN;
INSERT INTO visit (id, trip_id, facility_id, mode, driver_device_id, weather_json, expires_at)
VALUES ('visit-b-pick', 'trip-b-001', 1, 'pickup', 'device-car-001',
        '{"condition":"rain","temp_c":13}', datetime('now', '+1 day'));
INSERT INTO participant (visit_id, role, token, display_name, current_wait_spot_id, status)
VALUES ('visit-b-pick', 'driver',    'tok-b2-driver',    '아빠', NULL, 'none'),
       ('visit-b-pick', 'passenger', 'tok-b2-passenger', '엄마', 3,    'preparing');
INSERT INTO candidate (visit_id, purpose, access_point_id, wait_spot_id, connection_id, rank, score, reasons_json)
VALUES
  ('visit-b-pick', 'meet', 4, 3, 5, 1, 0.94, '["차가 있는 B2","엘리베이터 실내 연결","비 안 맞음"]'),
  ('visit-b-pick', 'meet', 2, 2, 3, 2, 0.80, '["지붕 있음","차를 빼야 함"]');
UPDATE visit
   SET confirmed_candidate_id = (SELECT id FROM candidate WHERE visit_id = 'visit-b-pick' AND access_point_id = 4),
       version = version + 1
 WHERE id = 'visit-b-pick' AND version = 1;
COMMIT;

SELECT id, mode, status, current_stage, version
  FROM visit WHERE trip_id = 'trip-b-001' ORDER BY created_at, rowid;

-- 외출 링크로 열면: 진행 중인 방문 (04의 Q11)
SELECT t.id AS trip_id, v.id AS visit_id, v.mode, v.status
  FROM trip t
  JOIN visit v ON v.trip_id = t.id
 WHERE t.share_token = 'tok-trip-b'
   AND v.status NOT IN ('ended', 'cancelled')
 ORDER BY v.created_at DESC, v.rowid DESC
 LIMIT 1;
```

→ trip-b-001에 방문 2개 (dropoff ended, pickup created). 외출 링크 → visit-b-pick, 만남 장소 지하주차장 B2 입구 / 2층 가족휴게실

B-7. 만나기

엄마가 휴게실에서 B2로 내려오고, 아빠가 확인하면 픽업 방문도 끝나요(version 3).

```
BEGIN;
UPDATE participant SET status = 'at_lobby', last_seen_at = datetime('now')
 WHERE token = 'tok-b2-passenger';
INSERT INTO stage_event (visit_id, stage, actor_id, request_id)
VALUES ('visit-b-pick', 'arrived', (SELECT id FROM participant WHERE token = 'tok-b2-driver'), 'req-b2-arrived'),
       ('visit-b-pick', 'boarded', (SELECT id FROM participant WHERE token = 'tok-b2-driver'), 'req-b2-boarded');
UPDATE visit SET status = 'ended', current_stage = 'ended', version = version + 1
 WHERE id = 'visit-b-pick' AND version = 2;
INSERT INTO destination_history (driver_device_id, facility_id, mode, last_candidate_id)
SELECT driver_device_id, facility_id, mode, confirmed_candidate_id FROM visit WHERE id = 'visit-b-pick';
COMMIT;

SELECT id, mode, status, version FROM visit WHERE trip_id = 'trip-b-001' ORDER BY created_at, rowid;
```

→ visit-b-pick: ended, version 3. 엄마 status at_lobby

시나리오 C. 가족 픽업 후 바로 다른 장소로

A처럼 엄마를 데리러 가고, 엄마는 ETA를 보고 딱 맞춰 나와요. 그다음 같이 미술관에 가는데 비가 와서 실내 주차장을 추천받아요. 방문 2개를 trip으로 묶어요.

단계	하는 일	결과
C-1	출발 전: 목적지 입력, 만남 장소 추천과 결정	visit-c-pick: 서문 승하차 구역 / 1층 서문 로비, version 2
C-2	동승자에게 링크 전달	링크가 열리고 is_closed = 0
C-3	운전자 이동, 동승자가 ETA 확인	passenger_instruction = wait (600초 > 도보 120초)
C-4	제시간에 만남	판정 go_now → 엄마 at_lobby → visit-c-pick ended, version 5
C-5	다른 장소 입력, 날씨에 따른 주차 자리 추천	추천 1순위 미술관 지하 주차장 (rain 점수 3), 2순위 야외 주차장 (0). 지하로 결정하고 출발(departed), version 2
C-6	주차 완료 (행복한 시간)	trip-c-001: visit-c-pick(pickup, ended) → visit-c-park(parking, ended, 미술관 지하 주차장)

C-1. 출발 전: 목적지 입력, 만남 장소 추천과 결정

외출(trip)을 만들고, A-1과 A-2의 앞부분을 한 번에 해요. 서문을 골라 version 2.

```
BEGIN;
INSERT INTO trip (id, driver_device_id, share_token)
VALUES ('trip-c-001', 'device-car-001', 'tok-trip-c');
INSERT INTO visit (id, trip_id, facility_id, mode, driver_device_id, weather_json, expires_at)
VALUES ('visit-c-pick', 'trip-c-001', 1, 'pickup', 'device-car-001',
        '{"condition":"cloudy","temp_c":16}', datetime('now', '+1 day'));
INSERT INTO participant (visit_id, role, token, display_name)
VALUES ('visit-c-pick', 'driver', 'tok-c-driver', '아빠');
INSERT INTO candidate (visit_id, purpose, access_point_id, wait_spot_id, connection_id, rank, score, reasons_json, excluded_reason)
VALUES
  ('visit-c-pick', 'meet', 1, 1,    1,    1,    0.86, '["가장 가까움","도보 2분"]', NULL),
  ('visit-c-pick', 'meet', 2, 2,    3,    2,    0.82, '["지붕 있음"]', NULL),
  ('visit-c-pick', 'meet', 3, NULL, NULL, NULL, NULL, NULL, '승하차 금지 구역');
UPDATE visit
   SET confirmed_candidate_id = (SELECT id FROM candidate WHERE visit_id = 'visit-c-pick' AND access_point_id = 1),
       version = version + 1
 WHERE id = 'visit-c-pick' AND version = 1;
COMMIT;
```

→ visit-c-pick: 서문 승하차 구역 / 1층 서문 로비, version 2

C-2. 동승자에게 링크 전달

엄마는 지금 서문 로비 근처에 있어요(current_wait_spot_id = 1).

```
INSERT INTO participant (visit_id, role, token, display_name, current_wait_spot_id, status)
VALUES ('visit-c-pick', 'passenger', 'tok-c-passenger', '엄마', 1, 'preparing');

SELECT p.role, v.id AS visit_id, v.status,
       CASE WHEN v.expires_at < datetime('now') OR v.status IN ('ended', 'cancelled')
            THEN 1 ELSE 0 END AS is_closed
  FROM participant p JOIN visit v ON v.id = p.visit_id
 WHERE p.token = 'tok-c-passenger';
```

→ 링크가 열리고 is_closed = 0

C-3. 운전자 이동, 동승자가 ETA 확인

차가 10분(600초) 거리예요. 엄마 화면의 판정은 '아직 기다리기'예요.

```
BEGIN;
INSERT INTO stage_event (visit_id, stage, actor_id, request_id)
VALUES ('visit-c-pick', 'pickup_started', (SELECT id FROM participant WHERE token = 'tok-c-driver'), 'req-c-start');
UPDATE visit SET status = 'moving', current_stage = 'pickup_started', version = version + 1
 WHERE id = 'visit-c-pick' AND version = 2;
INSERT INTO vehicle_update (visit_id, lat, lng, eta_sec, nav_result, nav_target_access_point_id)
VALUES ('visit-c-pick', 37.5320, 127.2050, 600, 'set', 1);
UPDATE visit SET last_lat = 37.5320, last_lng = 127.2050, last_eta_sec = 600,
                 last_nav_result = 'set', last_vehicle_at = datetime('now')
 WHERE id = 'visit-c-pick';
COMMIT;

SELECT last_eta_sec, walk_minutes * 60 AS walk_sec,
       CASE WHEN last_eta_sec IS NULL THEN 'no_location'
            WHEN last_eta_sec <= walk_minutes * 60 THEN 'go_now'
            ELSE 'wait' END AS passenger_instruction
  FROM v_visit_card WHERE visit_id = 'visit-c-pick';
```

→ passenger_instruction = wait (600초 > 도보 120초)

C-4. 제시간에 만남

차가 110초 거리가 되면 '이제 나오세요'(go_now). 서버가 arrival_imminent를 남기고(version 4), 엄마가 걸어 나와 딱 맞춰 만나면 방문을 닫아요(version 5).

```
BEGIN;
INSERT INTO vehicle_update (visit_id, lat, lng, eta_sec, nav_result, nav_target_access_point_id)
VALUES ('visit-c-pick', 37.5398, 127.2185, 110, 'set', 1);
UPDATE visit SET last_lat = 37.5398, last_lng = 127.2185, last_eta_sec = 110, last_vehicle_at = datetime('now')
 WHERE id = 'visit-c-pick';
INSERT INTO stage_event (visit_id, stage, source, request_id)
VALUES ('visit-c-pick', 'arrival_imminent', 'system', 'req-c-imminent');
UPDATE visit SET current_stage = 'arrival_imminent', version = version + 1
 WHERE id = 'visit-c-pick' AND version = 3;
COMMIT;

SELECT last_eta_sec, walk_minutes * 60 AS walk_sec,
       CASE WHEN last_eta_sec IS NULL THEN 'no_location'
            WHEN last_eta_sec <= walk_minutes * 60 THEN 'go_now'
            ELSE 'wait' END AS passenger_instruction
  FROM v_visit_card WHERE visit_id = 'visit-c-pick';

BEGIN;
UPDATE participant SET status = 'at_lobby', last_seen_at = datetime('now') WHERE token = 'tok-c-passenger';
INSERT INTO stage_event (visit_id, stage, actor_id, request_id)
VALUES ('visit-c-pick', 'arrived', (SELECT id FROM participant WHERE token = 'tok-c-driver'), 'req-c-arrived'),
       ('visit-c-pick', 'boarded', (SELECT id FROM participant WHERE token = 'tok-c-driver'), 'req-c-boarded');
UPDATE visit SET status = 'ended', current_stage = 'ended', version = version + 1
 WHERE id = 'visit-c-pick' AND version = 4;
INSERT INTO destination_history (driver_device_id, facility_id, mode, last_candidate_id)
SELECT driver_device_id, facility_id, mode, confirmed_candidate_id FROM visit WHERE id = 'visit-c-pick';
COMMIT;
```

→ 판정 go_now → 엄마 at_lobby → visit-c-pick ended, version 5

C-5. 다른 장소 입력, 날씨에 따른 주차 자리 추천

미술관(시설 2)으로 가는 parking 방문을 같은 trip_id로 만들어요. 아래 추천 쿼리는 weather_json이 rain이면 실내·지붕·비 안 맞는 길이 많은 곳을 먼저 보여줘요.

```
BEGIN;
INSERT INTO visit (id, trip_id, facility_id, mode, driver_device_id, weather_json, expires_at)
VALUES ('visit-c-park', 'trip-c-001', 2, 'parking', 'device-car-001',
        '{"condition":"rain","temp_c":15}', datetime('now', '+1 day'));
INSERT INTO participant (visit_id, role, token, display_name)
VALUES ('visit-c-park', 'driver', 'tok-c2-driver', '아빠');
COMMIT;

SELECT ap.id AS access_point_id, ap.name, ap.indoor, ap.has_roof, cn.walk_minutes, cn.covered,
       CASE WHEN json_extract(v.weather_json, '$.condition') = 'rain'
            THEN ap.indoor + ap.has_roof + COALESCE(cn.covered, 0) ELSE 0 END AS rain_score
  FROM visit v
  JOIN access_point ap  ON ap.facility_id = v.facility_id AND ap.type = 'parking' AND ap.active = 1
  LEFT JOIN connection cn ON cn.access_point_id = ap.id
 WHERE v.id = 'visit-c-park'
 ORDER BY rain_score DESC, cn.walk_minutes;

BEGIN;
INSERT INTO candidate (visit_id, purpose, access_point_id, wait_spot_id, connection_id, rank, score, reasons_json)
VALUES
  ('visit-c-park', 'parking', 6, 4, 7, 1, 0.95, '["비","지하 주차장","로비까지 실내 연결"]'),
  ('visit-c-park', 'parking', 5, 4, 6, 2, 0.60, '["입구 가까움","비 맞음"]');
INSERT INTO stage_event (visit_id, stage, actor_id, request_id)
VALUES ('visit-c-park', 'departed', (SELECT id FROM participant WHERE token = 'tok-c2-driver'), 'req-c2-departed');
UPDATE visit
   SET parking_candidate_id = (SELECT id FROM candidate WHERE visit_id = 'visit-c-park' AND access_point_id = 6),
       status = 'moving', current_stage = 'departed', version = version + 1
 WHERE id = 'visit-c-park' AND version = 1;
COMMIT;
```

→ 추천 1순위 미술관 지하 주차장 (rain 점수 3), 2순위 야외 주차장 (0). 지하로 결정하고 출발(departed), version 2

C-6. 주차 완료 (행복한 시간)

주차를 저장하고 방문을 닫아요(version 3). trip_id로 하루 외출 전체를 볼 수 있어요.

```
BEGIN;
INSERT INTO stage_event (visit_id, stage, actor_id, request_id)
VALUES ('visit-c-park', 'parked', (SELECT id FROM participant WHERE token = 'tok-c2-driver'), 'req-c2-parked');
UPDATE visit SET status = 'ended', current_stage = 'ended', version = version + 1
 WHERE id = 'visit-c-park' AND version = 2;
INSERT INTO destination_history (driver_device_id, facility_id, mode, last_candidate_id)
SELECT driver_device_id, facility_id, mode, parking_candidate_id FROM visit WHERE id = 'visit-c-park';
COMMIT;

SELECT v.id, v.mode, f.name AS facility, v.status, v.version, ap.name AS parked_at
  FROM visit v
  JOIN facility f ON f.id = v.facility_id
  LEFT JOIN candidate c      ON c.id  = v.parking_candidate_id
  LEFT JOIN access_point ap ON ap.id = c.access_point_id
 WHERE v.trip_id = 'trip-c-001'
 ORDER BY v.created_at, v.rowid;
```

→ trip-c-001: visit-c-pick(pickup, ended) → visit-c-park(parking, ended, 미술관 지하 주차장)

구조 들여다보기 명령

명령 (sqlite3 안에서)	보여주는 것
.tables	테이블 목록
.schema visit	테이블을 만든 CREATE 문 (제약 포함)
PRAGMA table_info(visit);	열 이름, 타입, NOT NULL, 기본값
PRAGMA foreign_key_list(candidate);	그 테이블의 외래키
SELECT * FROM v_visit_card;	방문 카드 전체
.quit	나가기 (파일 잠금 풀림)

다시 처음부터 하고 싶으면 sqlite3를 .quit으로 나간 뒤 '준비' 단계를 다시 실행하세요.